

Trabajo Práctico 2 (TP2) — Base de Datos II: Integridad, Transacciones y Concurrency

Proyecto Integrador: Food Store

Alumno / Repositorio: BaseDeDatos2 (UTN - Tecnicatura Universitaria en Programación a Distancia)

Índice General

1. [Parte 0 — Protocolo de Seguridad](#)
 2. [Parte 1 — Integridad Versionada \(Restricciones y Triggers\)](#)
 - [Script SQL de Restricciones](#)
 - [DUIA - Parte 1](#)
 3. [Parte 2 — Laboratorio de Concurrency e Informe](#)
 - [Informe de Concurrency](#)
 - [DUIA - Parte 2](#)
 4. [Parte 3 — Lectura Crítica de Scripts](#)
 - [Ejercicio de Lectura Crítica](#)
 - [DUIA - Parte 3](#)
-

Parte 0 — Protocolo de Seguridad

Este documento establece el procedimiento estándar y obligatorio para cualquier modificación de esquema o datos sobre las bases de datos del proyecto **Food Store**, garantizando la integridad de los entornos y previniendo pérdida de datos o incidentes operativos.

1. Copia de Trabajo (Aislamiento de Entorno)

Nunca se trabaja directamente sobre la base de datos principal de producción o datos críticos. Toda experimentación, script generado por IA o prueba estructural se ejecuta sobre una copia de trabajo dedicada (**foodstore_copia**).

- **Comando de creación de copia en PostgreSQL:**

```
createdb -U postgres -T foodstore foodstore_copia
```

- **Regla:** Si se requiere reiniciar el entorno de pruebas, se elimina la copia y se regenera desde la plantilla base:

```
dropdb -U postgres foodstore_copia  
createdb -U postgres -T foodstore foodstore_copia
```

2. Transacción de Prueba (**BEGIN ... ROLLBACK**)

Todo script que modifique datos (DML: **INSERT**, **UPDATE**, **DELETE**) o ejecute operaciones lógicas debe ejecutarse inicialmente dentro de una transacción de prueba.

- **Estructura obligatoria de ejecución:**

```
BEGIN;  
-- Ejecución del script propuesto / generado por IA  
-- (Sentencias SQL aquí)  
SELECT * FROM ...;  
-- Si el resultado es el esperado: COMMIT;  
-- Si hay anomalías o efectos no deseados: ROLLBACK;  
ROLLBACK;
```

- **Regla:** Ningún cambio de datos se confirma (**COMMIT**) sin haber inspeccionado previamente el estado intermedio mediante **ROLLBACK** o validación explícita de filas afectadas.

3. Respaldo Estructural (**pg_dump**)

Antes de aplicar cualquier cambio estructural (DDL: **ALTER TABLE**, **DROP TABLE**, migraciones complejas), se debe generar un respaldo físico independiente de la copia de trabajo.

- **Comando de respaldo en PostgreSQL:**

```
pg_dump -U postgres -d foodstore_copia -F c -b -v -f "backup_foodstore_$(date +%Y%m%d_%H%M%S).backup"
```

- **Regla:** Ante cualquier error estructural irreversible, se restaura desde el respaldo generado:

```
pg_restore -U postgres -d foodstore_copia -v "backup_foodstore_YYYYMMDD_HHMMSS.backup"
```

Parte 1 — Integridad Versionada (Restricciones y Triggers)

Script SQL de Restricciones

```
-- =====
-- Proyecto Integrador: Food Store (TP2 - Parte 1)
-- Archivo: restricciones_tp2.sql
-- Motor: PostgreSQL
-- Descripción: Implementación de restricciones de negocio mediante Triggers
--              y validaciones declarativas (CHECK).
-- =====

-- -----
-- Regla 1: Un cliente inactivo (activo = FALSE) no puede realizar pedidos nuevos.
-- Mecanismo: Trigger BEFORE INSERT en la tabla 'pedido'.
-- -----

CREATE OR REPLACE FUNCTION fn_verificar_cliente_activo()
RETURNS TRIGGER AS $$
DECLARE
    v_activo BOOLEAN;
BEGIN
    SELECT activo INTO v_activo
    FROM cliente
    WHERE id = NEW.cliente_id;
```

```

    IF NOT FOUND THEN
        RAISE EXCEPTION 'El cliente con ID % no existe.', NEW.cliente_id;
    END IF;

    IF v_activo IS FALSE THEN
        RAISE EXCEPTION 'No se puede crear el pedido: El cliente (ID: %) se
encuentra inactivo.', NEW.cliente_id;
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

DROP TRIGGER IF EXISTS trg_verificar_cliente_activo ON pedido;

CREATE TRIGGER trg_verificar_cliente_activo
    BEFORE INSERT ON pedido
    FOR EACH ROW
    EXECUTE FUNCTION fn_verificar_cliente_activo();

-- -----
-- Regla 2: Un pedido no puede contener cantidades iguales o menores a 0.
-- Mecanismo: Restricción declarativa CHECK (ya definida en DDL, documentada aquí).
-- -----
-- CONSTRAINT chk_detalle_cantidad_positiva CHECK (cantidad > 0) en pedido_detalle.

-- -----
-- Regla 3: Un producto inactivo (activo = FALSE) no puede ser utilizado en nuevos
pedidos.
-- Mecanismo: Trigger BEFORE INSERT en la tabla 'pedido_detalle'.
-- -----

CREATE OR REPLACE FUNCTION fn_verificar_producto_activo()
RETURNS TRIGGER AS $$
DECLARE
    v_activo BOOLEAN;
    v_nombre VARCHAR(100);
BEGIN
    SELECT activo, nombre INTO v_activo, v_nombre
    FROM producto
    WHERE id = NEW.producto_id;

    IF NOT FOUND THEN
        RAISE EXCEPTION 'El producto con ID % no existe.', NEW.producto_id;
    END IF;

    IF v_activo IS FALSE THEN
        RAISE EXCEPTION 'No se puede agregar el producto "%" (ID: %) al pedido
porque se encuentra inactivo.', v_nombre, NEW.producto_id;
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

DROP TRIGGER IF EXISTS trg_verificar_producto_activo ON pedido_detalle;

```

```
CREATE TRIGGER trg_verificar_producto_activo
BEFORE INSERT ON pedido_detalle
FOR EACH ROW
EXECUTE FUNCTION fn_verificar_producto_activo();
```

DUIA - Parte 1

Campo	Completar
Herramienta	OpenCode (modelo <code>google/gemini-3.5-flash-lite</code>)
Spec o prompt utilizado	1. Dentro de la tabla “cliente”, en la columna “activo”, un cliente con estado <code>false</code> no debe ser capaz de realizar pedidos nuevos. 2. Dentro de la tabla “pedido_detalle”, en la columna “cantidad”, si un detalle intenta ser creado con cantidad <code><= 0</code> no debe ser permitido. 3. Dentro de la tabla “producto”, en la columna “activo”, si el estado es <code>false</code> no se debe permitir que este producto sea usado en pedidos.
Qué generó	Un script SQL (<code>restricciones_tp2.sql</code>) conteniendo dos funciones PL/pgSQL con sus respectivos triggers (<code>trg_verificar_cliente_activo</code> y <code>trg_verificar_producto_activo</code>), además de la documentación/referencia para la restricción <code>CHECK</code> de cantidad en <code>pedido_detalle</code> .
Qué se aceptó	La lógica completa de validación mediante triggers para las reglas 1 y 3, incluyendo manejo de excepciones con mensajes claros (<code>RAISE EXCEPTION</code>), y la verificación de existencia (<code>IF NOT FOUND</code>).
Qué se modificó o descartó, y por qué	Se ajustaron los mensajes de error en los triggers para que devuelvan información detallada (IDs y nombres de productos/clientes) facilitando el diagnóstico operativo.
Verificación realizada	Prueba 1 (Cliente inactivo): Se intentó insertar un pedido para un cliente con <code>activo = FALSE</code>. Resultado: Rechazado por el trigger. Prueba 2 (Cantidad inválida): Se intentó insertar un detalle con <code>cantidad = 0</code>. Resultado: Rechazado por el <code>CHECK</code>.

Campo	Completar
	Prueba 3 (Producto inactivo): Se intentó agregar un producto con <code>activo = FALSE</code> . Resultado: Rechazado por el trigger.

Parte 2 — Laboratorio de Concurrency e Informe

Informe de Concurrency

Escenario 1: Lectura No Repetible (Non-Repeatable Read)

- Tabla involucrada: `producto`
- Comandos Sesión A (Read Committed):

```
BEGIN TRANSACTION ISOLATION LEVEL READ COMMITTED;  
SELECT precio FROM producto WHERE id = 1; -- Devuelve 2500.00
```

- Comandos Sesión B:

```
UPDATE producto SET precio = 2800.00 WHERE id = 1;  
COMMIT;
```

- Continuación Sesión A:

```
SELECT precio FROM producto WHERE id = 1; -- Devuelve 2800.00 (Cambió)  
COMMIT;
```

- Explicación de la IA: Bajo `READ COMMITTED`, transacciones concurrentes pueden modificar datos entre lecturas. Se evita con `REPEATABLE READ`.
- Verificación en el motor: Al repetir con `REPEATABLE READ`, la Sesión A siguió viendo `2500.00`. Conclusión confirmada.

Escenario 2: Espera por Bloqueo (Lock Waiting - `FOR UPDATE`)

- Tabla involucrada: `producto`

- **Comandos Sesión A:**

```
BEGIN;  
SELECT * FROM producto WHERE id = 1 FOR UPDATE;
```

- **Comandos Sesión B:**

```
BEGIN;  
UPDATE producto SET stock = stock - 1 WHERE id = 1; -- Queda en espera  
(blocking)
```

- **Liberación:** Tras el **COMMIT** de Sesión A, la Sesión B completa su ejecución inmediatamente.
- **Explicación de la IA y Conclusión:** **FOR UPDATE** adquiere bloqueo exclusivo a nivel de fila, previniendo condiciones de carrera. Verificado en el motor.

Escenario 3: Lectura Fantasma (Phantom Read)

- **Tabla involucrada:** **producto**
- **Comandos Sesión A (Read Committed):**

```
BEGIN TRANSACTION ISOLATION LEVEL READ COMMITTED;  
SELECT COUNT(*) FROM producto WHERE precio > 2000.00;
```

- **Comandos Sesión B:** Inserta un producto nuevo que cumple el predicado y hace **COMMIT**.
- **Continuación Sesión A:** Al repetir el **COUNT(*)**, arroja una fila más ("fantasma").
- **Verificación en el motor:** En PostgreSQL, bajo **REPEATABLE READ**, las lecturas fantasma se evitan gracias al MVCC estricto.

DUIA - Parte 2

Campo	Completar
Herramienta	OpenCode (modelo google/gemini-3.5-flash-lite)
Spec o prompt utilizado	Generar la estructura y redacción detallada para el informe_concurrencia.md reproduciendo los escenarios de Lectura No Repetible, Espera por Bloqueo (FOR UPDATE) y

Campo	Completar
	Lectura Fantasma sobre las tablas del proyecto Food Store en PostgreSQL.
Qué generó	La estructura completa del informe de concurrencia con los comandos exactos de Sesión A y Sesión B, explicaciones técnicas y verificación en el motor.
Qué se aceptó	La estructura metodológica y los bloques de comandos SQL específicos para las tablas producto y pedido de Food Store.
Qué se modificó o descartó, y por qué	Se ajustaron los valores y sentencias SQL para coincidir exactamente con los datos de carga inicial (seed.sql).
Verificación realizada	Ejecución práctica en dos sesiones paralelas (psql / DBeaver).

Parte 3 — Lectura Crítica de Scripts

Ejercicio de Lectura Crítica

1. Análisis del Script 1

- Script Original:

```
UPDATE funcion
SET activa = FALSE;
```

- **Efecto Real:** Afecta y desactiva **absolutamente todas** las filas de la tabla.
- **Por qué no coincide:** Carece de la cláusula **WHERE** para filtrar registros vencidos.
- **Versión Corregida:**

```
UPDATE funcion
SET activa = FALSE
WHERE fecha_funcion < CURRENT_DATE AND activa = TRUE;
```

2. Análisis del Script 2

- **Script Original:**

```
DELETE FROM categoria
WHERE id NOT IN (SELECT categoria_id FROM producto);
```

- **Efecto Real:** Si la subconsulta devuelve al menos un valor **NULL**, el operador **NOT IN** evalúa a **UNKNOWN** globalmente y **no elimina ninguna categoría**.
- **Por qué no coincide:** Vulnerabilidad ante valores **NULL** en subconsultas con **NOT IN**.
- **Versión Corregida (Recomendada con **NOT EXISTS**):**

```
DELETE FROM categoria c
WHERE NOT EXISTS (
    SELECT 1
    FROM producto p
    WHERE p.categoria_id = c.id
);
```

DUIA - Parte 3

Campo	Completar
Herramienta	OpenCode (modelo google/gemini-3.5-flash-lite)
Spec o prompt utilizado	Analizar críticamente los scripts provistos en la consigna (Script 1 sin WHERE y Script 2 con NOT IN vulnerable a NULLs) y redactar el análisis de impacto y su corrección para el archivo ejercicio_lectura_critica.md .
Qué generó	Explicación detallada del impacto de la ausencia de WHERE y del comportamiento tri-valorado de SQL con NULLs en NOT IN, junto con las alternativas usando NOT EXISTS.
Qué se aceptó	Todo el análisis teórico y las propuestas de corrección.
Qué se modificó o descartó, y por qué	Ninguna modificación requerida.
Verificación realizada	Validación conceptual mediante teoría de bases de datos relacionales.

